

Contents

Resumão Definitivo — Introdução a Sistemas Operacionais	1
(SENAC SP — todos os 8 capítulos, na régua pra prova)	1
Capítulo 1 — Fundamentos dos Sistemas Operacionais	1
Capítulo 2 — Arquitetura dos Sistemas Operacionais	2
Capítulo 3 — Arquitetura de Computadores (suporte ao SO)	2
Capítulo 4 — Processos e Threads	3
Capítulo 5 — Gerenciamento de Processos	4
Capítulo 6 — Gerenciamento de Memória	4
Capítulo 7 — Gerenciamento de Sistema de Arquivos	5
Capítulo 8 — Gerenciamento de E/S	5
QUIZ — 50 Perguntas pra Testar seu Aprendizado	6
GABARITO COMENTADO	7

Resumão Definitivo — Introdução a Sistemas Operacionais

(SENAC SP — todos os 8 capítulos, na régua pra prova)

Oi, linda! Preparei esse material te dando aquele resumo “sem gordura” de cada capítulo, com analogias pra você não esquecer NUNCA MAIS. No final tem 50 perguntas tipo quiz + gabarito comentado pra você testar se realmente aprendeu. Boa sorte na prova, você vai arrasar!

Capítulo 1 — Fundamentos dos Sistemas Operacionais

O que é um SO?

- É a camada de software que fica **entre o hardware e as aplicações**, coordenando tudo.
- Maziero resume em duas palavras: **abstração** (esconde a complexidade do hardware) e **gerência** (organiza quem usa o quê e quando).
- Machado e Maia: é “um conjunto de rotinas executado pelo processador” que controla o compartilhamento dos recursos (CPU, memória, E/S).
- **Analogia:** pensa no SO como aquela amiga organizadora de grupo de WhatsApp da família — ela não faz a festa, mas sem ela ninguém sabe quem leva o quê, que horas chega, nem como usar o espaço. Ela “abstrai” a bagunça e “gerencia” os recursos.

Linha do tempo (decore isso, cai muito em prova!):

- **1ª Plataforma:** mainframes, cartões perfurados, processamento em lote (*batch*), terminais, multiprogramação. Em 1969, Ken Thompson e Dennis Ritchie criam o **Unix** no Bell Labs.
- **2ª Plataforma:** PC (IBM-PC/x86, anos 80), Macintosh traz GUI e mouse, surgem DOS/Windows/macOS, cliente-servidor, internet, web (anos 90), movimento **open source**, **Linux** criado por **Linus Torvalds** (anos 90) e vira o SO mais usado do mundo nos anos 2010.
- **3ª Plataforma:** computação em **nuvem** (AWS, 2006, foi o primeiro provedor), + **mobilidade, big data & analytics, redes sociais e negócios** = os 4 pilares.

Tipos de SO (classificação prática do autor):

- **SO servidor:** roda 24x7, geralmente sem interface gráfica (mais segurança e economia de recursos), acesso remoto. É o SO do **back-end**.

- **SO desktop/móvel:** tem GUI, uso direto pelo usuário final. É o SO do **front-end**. Android e iOS entram aqui também.
-

Capítulo 2 — Arquitetura dos Sistemas Operacionais

Kernel (núcleo):

- É o "cérebro" que gerencia processos, memória, hardware, sistema de arquivos.
- Precisa rodar em **modo privilegiado (modo kernel)** — só ele pode executar certas instruções (proteger memória, mudar clock da CPU etc.). Programas comuns rodam em **modo usuário**.
- Para pedir algo ao kernel, os programas usam **system calls** (chamadas de sistema). Nos sistemas Unix, isso segue o padrão **POSIX**.
- **Analogia:** o modo kernel é tipo o crachá de acesso irrestrito do gerente numa empresa — só ele entra em certas salas (as instruções privilegiadas). Os funcionários comuns (programas em modo usuário) precisam "pedir autorização" (system call) pra conseguir algo daquela sala.

Espaço do usuário (userspace):

- Tarefas que não precisam de modo privilegiado (ex: impressão, rede) migraram pra fora do kernel arquitetura em camadas (kernel + userspace), usada pela maioria dos SOs atuais.

Interface com o usuário:

- **GUI** (gráfica) vs **CLI** (linha de comando, via um interpretador chamado **shell**).
- Shells: Bash (padrão em Linux/Unix, compatível com POSIX), PowerShell e prompt de comando (Windows), ZSH, Fish.
- **Importante:** Linux, tecnicamente, é só o **kernel**! O sistema completo com utilitários GNU (Bash, GCC etc.) é o **GNU/Linux**.
- Distribuições: família Debian/Ubuntu (apt, pacotes .deb) x família RedHat (dnf, pacotes .rpm).

Inicialização (boot):

- Sequência: **firmware** (BIOS antigo / **UEFI** atual, gravado em memória não volátil) verifica hardware **bootloader** (ex: GRUB) carrega o **kernel** kernel assume e inicia o userspace.
 - **Analogia:** é literalmente "se levantar puxando as alças das próprias botas" (daí o nome *boot*) — o sistema precisa "nascer sozinho", em etapas, até conseguir rodar por conta própria.
-

Capítulo 3 — Arquitetura de Computadores (suporte ao SO)

Peças do hardware:

- **CPU, memória principal** (RAM/DRAM — volátil), **memória secundária** (SSD/HD, Flash — não volátil), **memória cache** (SRAM, super rápida, entre CPU e RAM), **E/S** (módulos + periféricos) e **barramentos** (ex: USB).
- SoC ("system on chip") = vários componentes (CPU, GPU, NPU) no mesmo chip.

As 4 tarefas de gerenciamento do SO sobre o hardware (decore, é a espinha dorsal do curso inteiro!):

1. **Processador** escalonamento
2. **Memória** memória virtual

3. Sistema de arquivos

4. E/S device drivers

Concorrência e Interrupções (o coração deste capítulo):

- **Concorrência** = disputa “leal” pelos recursos do computador, permitindo que várias tarefas progridam (não precisa ser paralelismo de verdade).
 - **Interrupção (INT)**: mecanismo que faz o processador pausar o que está fazendo pra atender outra coisa urgente.
 - **A analogia clássica do próprio livro** (e olha que combina 100% com seu jeito de aprender!): você está estudando concentrada, esqueceu o celular no modo normal, ele **toca** — você marca onde parou, atende, resolve, e volta pro ponto exato de onde tinha parado. Isso é exatamente uma interrupção: o processo é pausado, o SO executa a **rotina de tratamento de interrupção (RTI/ISR)**, e depois retoma o processo de onde parou (**troca de contexto**).
 - **4 causas de interrupção**: E/S (tecla, clique, pacote de rede), **temporização** (tipo timer de cozinha), **falha de hardware** (impressora sem papel, erro de memória), **exceção** (falha de software — divisão por zero, acesso indevido à memória).
 - O temporizador garante que o SO **sempre retome o controle** da CPU — é o que possibilita a **preempção**.
-

Capítulo 4 — Processos e Threads

Processo = programa em execução.

- Informações-chave guardadas no **PCB (Process Control Block)**: **PID** (identificador único), nome, **usuário (UID)**, prioridade + métricas (tempo de CPU, memória usada, E/S, rede).
- No Linux, essas infos aparecem no pseudo-sistema de arquivos **/proc**.
- **Estados do processo**: criação **pronto** (na fila esperando vez) **execução** (rodando na CPU) pode voltar pra pronto, ir pra **espera/bloqueado** (esperando E/S ou timer) ou ser **terminado**. Só é finalizado estando em execução.
- Processos podem gerar **processos filhos**.
- **sudo**: permite rodar comandos como root sem precisar compartilhar a senha do root — mais seguro.

Threads:

- São “linhas de execução” **dentro** de um mesmo processo — pedaços de código independentes que rodam concorrentemente.
- **Analogia do próprio livro**: um editor de texto tem uma thread cuidando da digitação, outra checando ortografia, outra atualizando a tela, outra salvando em segundo plano. Se não fossem threads separadas, salvar o arquivo travaria a digitação!
- No Unix, threads são chamadas de **lightweight process (LWP)**.

Canalização e redirecionamento:

- Canais padrão: **stdin** (entrada), **stdout** (saída), **stderr** (erro).
 - Redirecionadores: **<** (entrada), **>** (saída, sobrescreve), **>>** (acrescenta no final).
 - **Pipe** | conecta a saída de um processo à entrada de outro. **&&** (executa só se o anterior deu certo) e **||** (executa só se o anterior falhou).
 - **Foreground** (interage com o usuário) x **background** (roda sem interação direta).
-

Capítulo 5 — Gerenciamento de Processos

Escalonamento (scheduling): define qual processo/thread usa a CPU e quando.

- **Time-sharing:** CPU revezada entre threads em fatias de tempo (**quantum/time slice**).
- **Preempção:** a thread em execução é "convidada a se retirar" e volta pra fila de pronto.

Algoritmos de escalonamento:

- **FIFO/FCFS:** executa na ordem de chegada.
- **SJF (Shortest Job First):** prioriza quem tem menor tempo estimado.
- **Round-Robin:** versão preemptiva (por tempo) do FIFO — todo mundo ganha sua fatia.
- **Round-Robin por prioridades:** combina justiça do Round-Robin + importância das prioridades. É o que o **Windows** usa (32 níveis de prioridade em 6 grupos).
- **Aging:** evita que processo de baixa prioridade fique esquecido pra sempre, promovendo-o aos poucos.
- **Prioridade dinâmica:** equilibra processos **I/O-bound** (interativos, usam pouco quantum, ganham prioridade ao voltar da espera) com **CPU-bound** (usam a CPU por mais tempo).
- **Linux:** usa **Multilevel Feedback Queue** (múltiplas filas conforme a prioridade); processos "normais" usam o **CFS (Completely Fair Scheduler)**, ajustado pelo **niceness** (-20 = mais prioritário, +19 = menos prioritário, 40 níveis usados na prática).

Deadlocks (impasses):

- Ocorre quando dois (ou mais) processos ficam esperando recursos um do outro pra sempre (processo A tem X e quer Y; processo B tem Y e quer X).
 - **Analogia:** cruzamento com semáforo quebrado, onde ninguém cede e todo mundo trava.
 - Conceitos-chave: **exclusão mútua** (só um processo acessa o recurso por vez), **seção crítica** (trecho que acessa recurso compartilhado).
 - Mecanismos pra evitar: **semáforos** (controlam acesso a recursos compartilhados) e **sistemas de mensageria** (comunicação indireta via buffer/caixa de mensagens).
-

Capítulo 6 — Gerenciamento de Memória

Problemas clássicos de particionamento:

- **Fragmentação interna:** sobra espaço desperdiçado *dentro* de uma partição alocada.
- **Fragmentação externa:** há espaço livre total suficiente, mas não contíguo.

Memória virtual (o assunto mais denso do capítulo):

- Técnica de **virtualização** que dissocia o endereço lógico (do programa) do endereço físico (real na RAM), permitindo que cada processo "enxergue" mais memória do que realmente existe, com exclusividade.
- A tradução lógico-físico é feita pela **MMU** (Memory Management Unit), usando **paginação**: memória lógica dividida em **páginas**, memória física em **frames**, e uma **tabela de páginas** faz o mapeamento.
- **Paginação sob demanda:** só carrega na memória física o trecho que realmente está sendo usado no momento.
- **Page fault:** ocorre quando a página não está na memória física.
- **Minor:** resolve só ajustando mapeamento (rápido).
- **Major:** precisa buscar a página na memória secundária (mais lento).
- **Working set** = conjunto de páginas mapeadas pro processo naquele momento.

Swap:

- Usa a memória secundária como "extensão" da RAM quando ela não é suficiente.
- Windows chama de **pagefile**; Unix/Linux de **swapfile**.

- **Page out** (tira da RAM, joga no disco) / **page in** (traz de volta).
- Trade-off: resolve o problema de tamanho, mas é bem mais lento que a RAM.

OOM Killer (Out of Memory killer):

- Quando o Linux está criticamente sem memória, ele "sacrifica" (mata) processos pra manter o sistema de pé. (nome dramático, mas sem drama de verdade, calma!)

Gerenciamento de pacotes:

- apt (Debian/Ubuntu) x dnf (RedHat/Fedora/Amazon Linux/Oracle Linux) x zypper (Suse).
 - Fluxo: consulta repositório checa dependências baixa e instala atualiza documentação (manpages).
 - Formatos modernos multiplataforma: **Snapcraft** e **Flatpak**.
-

Capítulo 7 — Gerenciamento de Sistema de Arquivos

A analogia da biblioteca (a base de tudo neste capítulo): um sistema de arquivos é como o catálogo de uma biblioteca gigante — sem ele, seria impossível achar um livro (arquivo) específico entre milhares espalhados nas prateleiras (dispositivos de armazenamento).

- **Arquivo:** dados nomeados + metadados (dono, datas, tamanho, permissões).
 - Estrutura em **árvore** (diretórios/subdiretórios) e **caminho (path)** identifica unicamente o arquivo.
 - **Bloco:** menor unidade de armazenamento (comumente 4 KB) — todo disco é um "dispositivo de bloco".
 - **Inode:** estrutura de índice que aponta pros blocos que formam o arquivo.
 - **Permissões Unix/Linux:** r (leitura), w (escrita), x (execução) para **usuário**, **grupo** e **outros**. Notação octal: rw-r--r-- = **644**; rwxr-xr-x = **755**.
 - **SETUID (bit "s"):** faz o programa rodar com o usuário dono do arquivo (não quem executou) — é como o passwd consegue trocar senha mesmo sendo um comando "comum".
 - **ACL (Access Control List):** mecanismo mais granular, padrão do Windows.
 - **Journaling:** registra alterações *antes* de aplicá-las no disco, protegendo contra corrupção em caso de queda de energia/travamento.
 - **PATH:** variável de ambiente com a lista de diretórios onde o sistema procura o executável quando você digita um comando.
 - Comandos básicos: mkdir, rmdir, touch, cp, mv, rm (-R para recursivo).
 - Exemplos de sistemas de arquivos: **EXT** (Linux), **NTFS** (Windows), **FAT32**.
-

Capítulo 8 — Gerenciamento de E/S

- Kernel não fala diretamente com o periférico: ele conversa com o **módulo de E/S** (controlador), que aí sim se comunica com o periférico via **device drivers** (tradutores especializados pra cada hardware).
- **DMA (Direct Memory Access):** permite transferência de dados entre memória e dispositivo **sem** passar pela CPU o tempo todo — libera a CPU pra outras tarefas. Muito usado em memória secundária.
- **DCA (Direct Cache Access):** variação do DMA, só que direto pra memória cache — comum em placas de rede rápidas.
- **Buffering:** área de memória temporária que equilibra velocidades diferentes entre dispositivos.
- **Dispositivos de bloco** (SSD, HD, pendrive) x **dispositivos de caractere** (teclado, impressora, terminal) — no ls -l, aparecem como b e c respectivamente.

- **Redes:** placa de rede = **NIC**; protocolos TCP/UDP/IP; **sockets** identificam processos que trocam dados pela rede; **sistemas distribuídos** (ex: nuvem) usam **middleware** como camada de comunicação entre aplicações.
- Comandos úteis de rede: `ip addr`, `ip link`, `ip route`, `ssh`, `ping`, `ss`, `firewalls` (`nftables`, `ufw`, `firewalld`).
- **Shell scripts:** automação de tarefas.
- Começam com **shebang**: `#!/bin/bash`.
- `#` = comentário. `echo` exibe mensagem, `read` lê entrada do usuário.
- Usam estruturas condicionais (`if/elif/else`) e laços (`while`).
- Precisam de permissão de execução: `chmod +x script`.

QUIZ — 50 Perguntas pra Testar seu Aprendizado

*Dica: responda tudo primeiro, só depois confira o gabarito lá embaixo! Assim você sabe de verdade o que já sabe *

Capítulo 1

1. Segundo Maziero, quais são as duas palavras-chave que sintetizam os objetivos de um SO?
2. Em qual período histórico os primeiros computadores eletrônicos digitais surgiram?
3. Quem criou o Unix, e em que década?
4. Quem criou o Linux?
5. Quais são os quatro pilares da "terceira plataforma computacional"?
6. Qual a diferença prática entre um SO servidor e um SO desktop?

Capítulo 2 7. O que é uma system call (chamada de sistema)? 8. Qual é o nome do padrão técnico que garante portabilidade das system calls nos sistemas Unix? 9. Tecnicamente, o que é "Linux": o sistema operacional completo ou apenas o kernel? 10. Qual é a shell mais popular nos sistemas Unix/Linux? 11. Qual é a ordem correta do processo de boot? 12. O que indica o caractere final `#` no prompt do Bash, em vez de `$`?

Capítulo 3 13. Qual é a diferença entre memória principal (RAM) e memória secundária em termos de volatilidade? 14. Quais são as quatro tarefas de gerenciamento do SO sobre o hardware? 15. O que é concorrência, segundo o material? 16. Cite as quatro causas/classes de interrupção. 17. O que é a RTI (rotina de tratamento de interrupção)? 18. O que garante que o SO sempre reassuma o controle da CPU de um processo? 19. O que significa a sigla DRAM e a que tipo de memória ela se refere?

Capítulo 4 20. O que é armazenado no PCB (Process Control Block)? 21. Quais são os três estados principais de um processo (além de criação e término)? 22. O que é uma thread e qual é o nome dado a ela nos sistemas Unix? 23. Para que serve o comando `sudo`? 24. O que faz o operador `|` (pipe) entre dois comandos? 25. Qual é a diferença entre `&&` e `||` ao encadear comandos?

Capítulo 5 26. O que é o "quantum" (ou time slice) no escalonamento? 27. Qual algoritmo de escalonamento é uma versão preemptiva (por tempo) do FIFO? 28. O que é o mecanismo de "aging" no escalonamento? 29. Qual escalonador o Linux usa para processos da classe "normal"? 30. O que é um deadlock? 31. Cite dois mecanismos usados para evitar deadlocks. 32. Para que serve o comando `nice`?

Capítulo 6 33. Qual é a diferença entre fragmentação interna e fragmentação externa? 34. Qual componente de hardware é responsável por traduzir endereços lógicos em físicos? 35.

O que é uma "página" e o que é um "frame" na memória virtual paginada? 36. Qual é a diferença entre um minor page fault e um major page fault? 37. O que é o mecanismo de swap e como ele é chamado no Windows e no Linux? 38. O que faz o OOM Killer no Linux? 39. Qual ferramenta de gerenciamento de pacotes é usada nas distribuições derivadas da RedHat?

Capítulo 7 40. O que é um inode? 41. O que significa a permissão octal 644 em um arquivo Unix/Linux? 42. O que é o bit SETUID e para que ele é usado (dê um exemplo)? 43. O que é journaling em um sistema de arquivos? 44. Para que serve a variável de ambiente PATH? 45. Cite dois exemplos de sistemas de arquivos mencionados no material.

Capítulo 8 46. Qual é a função de um device driver? 47. O que é o DMA (Direct Memory Access) e qual sua principal vantagem? 48. Qual é a diferença entre um dispositivo de bloco e um dispositivo de caractere? 49. O que é um socket, no contexto de redes? 50. Com quais caracteres um shell script deve obrigatoriamente começar, e como essa sequência é chamada?

GABARITO COMENTADO

1. **Abstração e gerência.** O SO esconde a complexidade do hardware (abstração) e organiza o uso dos recursos (gerência).
2. **Segunda Guerra Mundial** — foi nesse contexto que surgiram os primeiros computadores eletrônicos digitais.
3. **Ken Thompson e Dennis Ritchie**, no **Bell Labs**, no **fim da década de 1960**.
4. **Linus Torvalds**, cientista de computação finlandês, no começo da década de 1990.
5. **Nuvem (cloud computing), mobilidade, big data & analytics, e redes sociais e negócios.**
6. **SO servidor:** roda 24x7, geralmente sem GUI (interface textual), acessado remotamente, é o SO do back-end. **SO desktop:** tem GUI, operado diretamente por um usuário final (teclado/mouse/touch), é o SO do front-end.
7. **É o mecanismo usado por um programa para solicitar um serviço ao kernel** (ex: criar processo, ler/gravar arquivo, alterar status).
8. **POSIX** (Portable Operating System Interface), definido pelo IEEE e ratificado pela ISO.
9. **Apenas o kernel.** O sistema completo com os utilitários GNU é chamado de GNU/Linux.
10. **Bash** (Bourne Again Shell) — mais usada no meio corporativo e totalmente compatível com o POSIX.
11. **Firmware (BIOS/UEFI) verificação do hardware bootloader carregamento do kernel kernel inicia o userspace.**
12. Indica que o usuário logado é o **administrador (root)** em sistemas Unix/Linux.
13. **RAM (memória principal) é volátil** — perde os dados sem energia. **Memória secundária é não volátil** (ou, no caso de discos magnéticos antigos, mantém os dados mesmo sem energia de forma mecânica/magnética) — mantém os dados mesmo desligada.
14. **Gerenciamento do processador (escalonamento), gerenciamento de memória, gerenciamento de sistema de arquivos e gerenciamento de E/S.**
15. **A possibilidade de compartilhar os recursos do computador entre vários processos de forma "leal"**, permitindo que todos progridam — pode haver concorrência sem paralelismo.
16. **E/S, temporização, falha de hardware e exceção (falha de software).**
17. **É a rotina do próprio kernel que trata a interrupção** — também chamada de ISR (Interrupt Service Routine) em inglês.
18. **Um temporizador (timer)** disparado pelo kernel: se ele zerar sem o SO ter reassumido o controle por outro motivo, gera uma interrupção que força a retomada do controle.

19. **DRAM = Dynamic RAM (RAM dinâmica)** — é o tipo de semicondutor usado na **memória principal**.
20. **PID, nome do processo, usuário (UID), prioridade e métricas de uso** (tempo de CPU, memória, E/S, rede, entre outros).
21. **Execução, pronto e espera/bloqueado**.
22. **Thread é uma "linha de execução" independente dentro de um mesmo processo**; nos sistemas Unix, é chamada de **lightweight process (LWP)**.
23. **Permite executar comandos como usuário root (administrador) sem precisar compartilhar ou digitar a senha do root** — mais seguro.
24. **Conecta a saída (stdout) de um processo diretamente à entrada (stdin) de outro processo**.
25. **&& executa o segundo comando somente se o primeiro tiver sido bem-sucedido; || executa o segundo somente se o primeiro tiver falhado**.
26. **É a "fatia de tempo" máxima que uma thread pode ficar no estado de execução** antes de ser preemptada e voltar pra fila de pronto.
27. **Round-Robin**.
28. **É um mecanismo que promove temporariamente a prioridade de threads que estão há muito tempo sem executar**, evitando que fiquem "esquecidas" (inanição).
29. **CFS — Completely Fair Scheduler** (escalonamento completamente justo), ajustado pelo parâmetro *niceness*.
30. **É quando dois ou mais processos ficam esperando indefinidamente por recursos que estão alocados um ao outro**, travando o sistema (também chamado de "im-passe").
31. **Semáforos** (controlam acesso a recursos compartilhados) e **sistemas de mensageria** (comunicação indireta via buffer).
32. **Executar um processo já definindo uma prioridade diferente da padrão** (usa o parâmetro *niceness*). Para mudar a prioridade de um processo já em execução, usa-se o *renice*.
33. **Fragmentação interna**: espaço desperdiçado *dentro* de uma partição já alocada (partição maior que o programa). **Fragmentação externa**: há espaço livre total suficiente na memória, mas não de forma contígua para alocar um novo processo.
34. **A MMU (Memory Management Unit)**, com apoio do SO.
35. **Página** é o bloco de tamanho fixo na memória **lógica** (do programa); **frame** é o bloco correspondente na memória **física**.
36. **Minor page fault**: resolvido só ajustando informações de mapeamento, sem acessar a memória secundária (mais leve). **Major page fault**: precisa buscar a página na memória secundária (mais pesado, envolve E/S).
37. **Swap** é o uso da memória secundária como extensão da memória principal quando ela não é suficiente. No **Windows** é chamado de **pagefile** (arquivo de paginação); no **Unix/Linux**, de **swapfile** (arquivo/partição de swap).
38. **Ele "sacrifica" (finaliza) processos de forma forçada quando o sistema está criticamente sem memória**, para manter a estabilidade geral em vez de deixar o sistema todo travar/reiniciar.
39. **dnf** (usado por RedHat, Fedora, Amazon Linux e Oracle Linux).
40. **É a estrutura de índice (index-node) que aponta para os blocos de dados que compõem um arquivo** no sistema de arquivos.
41. **Equivale à permissão rw-r--r--**: o dono (u) pode ler e escrever; grupo (g) e outros (o) só podem ler.
42. **É um bit especial (representado por "s" no lugar do "x") que faz o programa ser executado com as permissões do usuário dono do arquivo**, e não do usuário que o executou. Exemplo clássico: o comando *passwd*, que precisa de acesso privilegiado para alterar a senha de um usuário comum.
43. **É o registro das alterações no sistema de arquivos antes de aplicá-las de fato ao disco**, garantindo consistência dos dados em caso de falhas (queda de energia, trava-

mentos).

44. **Ela contém a lista de diretórios onde o SO procura o arquivo executável quando um comando é digitado**, evitando que seja preciso digitar o caminho completo toda vez.
45. **EXT** (usado no Linux) e **NTFS** (usado no Windows) — também vale citar FAT32.
46. **Traduzir os comandos genéricos do kernel para os comandos específicos que cada hardware/periférico entende** (e vice-versa) — é o “tradutor” entre o SO e o dispositivo.
47. **É uma técnica que permite a transferência de dados diretamente entre memória e dispositivo de E/S, sem que a CPU precise mediar todo o processo**. Vantagem: libera a CPU para outras tarefas, aumentando o desempenho.
48. **Dispositivo de bloco** transfere dados em blocos de tamanho fixo (SSD, HD, pendrive). **Dispositivo de caractere** transfere um caractere (ou palavra) por vez (teclado, impressora, terminal).
49. **É o mecanismo que identifica um processo específico que está enviando/recebendo dados através da rede**, permitindo a comunicação via protocolos como TCP/UDP.
50. **Com #! (shebang), seguido do caminho do interpretador** — por exemplo, `#!/bin/bash`.

Dica final: se alguma pergunta você errou, volta no capítulo correspondente do resumo e lê de novo a analogia — é isso que vai grudar na memória de verdade na hora da prova. Você consegue!